STREAMDECK

Multi-Streamer OBS Overlay Manager for Kick.com

Product Requirements Document

Co-op streaming • Kick webhooks • Auth system • Node.js + Vercel + Supabase

Version 3.0 — March 2026

Confidential — For Development Team

1. Executive Summary

StreamDeck is a self-hosted web application for Kick.com streamers that provides real-time overlay management, automatic event-driven alerts, a subathon countdown timer, a multi-tier sub goal system, a roulette wheel, and a full undo/revert engine. It is designed for co-op streaming, where two streamers share a countdown and goal system across their independent overlay outputs.

What makes this different

- Two-streamer co-op mode: shared countdown and goals across independent OBS overlays
- Authentication via Kick OAuth 2.1 with mod role support
- Every automated and manual action is reversible via a linked undo system
- GPU-composited CSS-only animations that do not compete with game capture in OBS
- Deployable on Vercel (free tier) with Supabase for persistence

> *This PRD supersedes PRD v2.0 and Features Spec v3.0. It adds: authentication system, multi-streamer co-op mode, per-user overlay isolation, cross-streamer notifications, and the complete database schema. The agent.md file accompanies this document and provides exact build instructions for AI coding agents.*

Target users

Role	Who	What they do
Streamer	Kick.com content creator	Logs in with Kick OAuth. Manages their overlay, countdown, goals. Creates co-op sessions. Invites mods.
Mod	Trusted assistant	Logs in with Kick OAuth. Controls assigned streamer's dashboard. Cannot create co-op sessions.
Viewer	Stream audience	No login. Their subs/follows/gifts trigger automatic events. Sees overlay in the stream.

2. System Architecture

The system consists of four layers: the Kick platform (external), a Node.js server, client screens (dashboard + overlay per user), and a Supabase database for persistence and real-time sync.

2.1 Three screens per streamer

Screen	URL	Purpose
Login	/login	Kick OAuth button + mod login. Public page.
Dashboard	/dashboard	Control panel. Requires auth. Streamer or mod session.
Overlay	/overlay/:userId	OBS browser source. NO auth. Transparent background. Per-user.

> *Each streamer gets their own overlay URL. OBS on Streamer A's machine points to /overlay/streamerA-uuid. OBS on Streamer B's machine points to /overlay/streamerB-uuid. The overlays are completely independent except for shared co-op state (countdown + goals).*

2.2 Server routes

Method	Route	Description
GET	/login	Login page (public)
GET	/api/auth/kick	Initiates Kick OAuth 2.1 flow with PKCE
GET	/api/auth/callback	Kick OAuth callback, exchanges code for tokens
POST	/api/auth/logout	Destroys session
GET	/dashboard	Dashboard SPA (requires auth)
GET	/overlay/:userId	OBS overlay output (public, per-user)
POST	/api/webhook/:userId	Kick webhook receiver (per-user endpoint)
WS	/ws	WebSocket hub for real-time communication

2.3 Tech stack

Layer	Technology	Rationale
Server	Node.js 20+ / Express	Vercel-compatible, fast WS support
WebSocket	ws library	Lightweight, no Socket.IO overhead
Auth	Kick OAuth 2.1 + PKCE + express-session	Native Kick login, session-based auth
Database	Supabase (PostgreSQL)	Free tier, Realtime subs, row-level security
Overlay	Single HTML + vanilla JS + CSS	OBS browser source: no build step allowed

Dashboard	React via CDN or vanilla JS	Complex state mgmt; CDN avoids build tools
Deploy	Vercel + Supabase	Free tier, auto-deploy, serverless webhooks

3. Authentication System

The login system uses Kick's official OAuth 2.1 with PKCE for streamer authentication. Mods also log in via Kick OAuth but are assigned to streamers through an internal permission system.

3.1 Kick OAuth 2.1 flow

1. User clicks "Login with Kick" on the login page
2. Server generates PKCE code_verifier + code_challenge (S256)
3. Server redirects to https://id.kick.com/oauth/authorize with client_id, redirect_uri, scopes, code_challenge, state
4. User approves on Kick's authorization page
5. Kick redirects back to /api/auth/callback with authorization code
6. Server exchanges code for tokens via POST https://id.kick.com/oauth/token
7. Server fetches user profile via GET https://api.kick.com/public/v1/users
8. Server creates or updates user in Supabase users table
9. Server subscribes to Kick webhook events for this user
10. Server creates express-session and redirects to /dashboard

> *OAuth scopes required: user:read, channel:read, events:subscribe, chat:write. The events:subscribe scope is critical — without it, the server cannot subscribe to webhook events for the user's channel.*

3.2 Roles and permissions

Role	Can do	Cannot do
Streamer	Full control of own overlay, countdown, goals, roulette. Create co-op sessions. Invite mods. Manage presets.	Control other streamers' overlays
Mod	Control assigned streamer's overlay toggles, alerts, countdown manual controls. Skip/replay alerts.	Create co-op sessions. Change streamer's auth settings. Delete configs.

3.3 Mod assignment flow

11. Streamer opens dashboard settings, enters mod's Kick username
12. Server looks up the Kick user in the database (or stores the username for future login)
13. Server creates entry in mod_assignments table
14. When mod logs in via Kick OAuth, server checks mod_assignments
15. Mod's dashboard shows a streamer selector dropdown listing assigned streamers
16. Mod selects a streamer and controls their overlay as if it were their own (with restricted permissions)

3.4 Login page design

The login page is a simple, dark-themed page with two paths:

- Primary: "Login with Kick" button (large, Kick green) that initiates OAuth flow

- Secondary: text below explaining that mods also use the "Login with Kick" button and will be routed to their assigned streamer's dashboard automatically

No separate registration flow — accounts are created automatically on first Kick OAuth login. The role is determined by whether the user exists in mod_assignments.

4. Co-op Streaming System

This is the flagship differentiator. Two streamers doing a co-op stream can link their StreamDeck instances so they share a countdown timer and sub goal system, while maintaining completely independent overlays for everything else.

4.1 How co-op works

17. Streamer A opens their dashboard and creates a co-op session (names it, e.g., "Horror Marathon")
18. Streamer A invites Streamer B by entering their Kick username
19. Streamer B sees an invite notification on their dashboard and accepts
20. Both streamers toggle which features to share: countdown, goals, or both
21. A shared state layer is created in the coop_sessions table
22. Both streamers' overlays now show the shared countdown and/or goal bar
23. Events from either streamer's channel affect the shared state

> *Key principle: overlays are always independent. Co-op only shares the countdown value and goal counter. Alerts, element positions, styles, roulette, and all other overlay elements remain per-streamer. Each streamer's OBS points to their own /overlay/:userId URL.*

4.2 Event flow with co-op active

Scenario: Viewer subs to Streamer A's channel. Co-op countdown and goals are both enabled.

Step	What happens	Where it appears
1	Kick sends webhook to /api/webhook/streamerA-uuid	Server only
2	Server fires sub alert for Streamer A	Streamer A's overlay ONLY
3	Server adds 30s to shared countdown	BOTH overlays + BOTH dashboards
4	Server increments shared goal counter by 1	BOTH overlays + BOTH dashboards
5	If goal is reached: fire goal-reached notification	BOTH overlays (text: "via Streamer A's viewers")
6	Push undo entries for both streamers	Both dashboards' undo panels

4.3 What is shared vs personal

Personal (per overlay)	Shared (co-op session)
Alert box (sub, follow, gift alerts)	Countdown timer value + config
Webcam frame, social bar, chat ticker	Goal list + current count + completion status
Element positions and styles	Cross-streamer notifications
Scene presets	(future: shared roulette)

Roulette wheel	
Undo history (personal actions)	

4.4 Cross-streamer notifications

When a shared goal is reached due to events on Streamer A's channel, Streamer B's overlay displays a notification that credits Streamer A. The notification format:

> *"Goal 'Play a horror game' reached! (via StreamerA's viewers)" — This appears on Streamer B's overlay with a celebration animation. Streamer A sees the standard goal-reached celebration. The notification clearly attributes the achievement to the correct source.*

The same principle applies to countdown updates. When time is added from Streamer A's sub event, Streamer B's overlay shows "+30s from viewer_joe (via StreamerA)" so both audiences understand the source.

4.5 Co-op session management (dashboard)

- Create session: name + invite Kick username
- Pending invites panel (for invited streamer)
- Active session panel: shows co-op partner's name, avatar, online status
- Feature toggles: share countdown (on/off), share goals (on/off)
- Leave session: returns to personal countdown/goals
- End session (creator only): dissolves the co-op for both streamers

5. Countdown Timer

A server-side timer that ticks down every second. Auto-extends when Kick sub events arrive. In co-op mode, the timer is shared across linked streamers. Fully configurable and revertable.

5.1 Configuration

Setting	Type	Description
startTime	HH:MM:SS	Initial countdown value when started (default 01:00:00)
timePerSub	seconds	Seconds added per new subscription (default 30)
timePerGiftedSub	seconds	Seconds per gifted sub, multiplied by gift count (default 30)
timePerFollow	seconds	Seconds per follow, 0 to disable (default 0)
timePerKicksGift	seconds	Seconds per Kicks native gift (default 15)
maxCap	HH:MM:SS	Timer cannot exceed this value (default 24:00:00)
minFloor	seconds	Timer stops at this value (default 0)
autoStart	boolean	Start when stream goes live (default false)

5.2 Co-op countdown behavior

When co-op countdown is enabled, both streamers' overlays display the same timer value from the coop_sessions.shared_countdown state. Any sub/gift event on either channel adds time to the shared timer. The server broadcasts the update to all co-op members' overlays simultaneously. The "+Xs from username" chip on each overlay includes "(via StreamerX)" when the event came from the co-op partner's channel.

5.3 Overlay rendering

- Large monospaced HH:MM:SS display, configurable position/size/color
- +Xs from username chips slide up below timer, hold 3 seconds, fade out
- Co-op chips include "(via StreamerX)" attribution
- Last 60 seconds: digits pulse red with subtle scale animation
- Zero state: "TIME'S UP" with dramatic scale-pop

6. Sub Goal System

A sequential ladder of milestones with percentage tracking. In co-op mode, goals are shared and events from either channel increment the counter.

6.1 Goal structure

Each goal has: id, name, target sub count, reward description. Goals are ordered sequentially. In cumulative mode, reaching goal 1 at 20 subs means you're already 20/50 toward goal 2.

6.2 Configuration

Setting	Type	Description
goals list	Array	Each: name, target count, reward text. Drag to reorder.
countMode	Dropdown	Cumulative (subs count toward all) or Per-goal (resets per goal)
countSources	Multi-select	Which Kick events increment: new subs, gifted subs, renewals, Kicks gifts
giftMultiplier	Number	1 gifted sub = X goal increments (default 1)
showCompleted	Toggle	Show completed goals in overlay or hide them
autoCelebrate	Toggle	Fire celebration alert on goal completion

6.3 Co-op goals behavior

When co-op goals are enabled, both streamers see the same goal list and progress. Sub events from either channel increment the shared counter. When a goal is reached:

- The streamer whose viewer caused the completion sees the standard celebration
- The co-op partner sees: "Goal 'X' reached! (via PartnerName's viewers)"
- Both overlays update the progress bar/ladder simultaneously

6.4 Overlay display modes

Single bar: Horizontal progress bar showing the active goal. Name on left, count on right. Fill animates smoothly. On completion: bar flashes, next goal slides in.

Ladder: Vertical stack of all goals. Active goal has colored fill. Completed goals show checkmark. Upcoming goals grayed out.

7. Roulette Wheel

A visual spin-the-wheel triggered manually or automatically. Segment actions can affect the countdown and goals, propagating through co-op if active.

7.1 Segment action types

Action	Behavior	Undoable
Display only	Shows text on overlay for X seconds.	Yes
Add countdown	Adds seconds to timer. Propagates to co-op.	Yes
Remove countdown	Subtracts from timer. Propagates to co-op.	Yes
Add goal progress	Adds N to goal counter. Propagates to co-op.	Yes
Trigger alert	Fires custom alert on this streamer's overlay only.	Yes
Switch preset	Changes scene preset on this streamer's overlay only.	Yes

> *Co-op propagation: if a roulette spin lands on "+5:00 to countdown" and co-op countdown is active, the time is added to the SHARED countdown, affecting both streamers. The spin animation and result display only on the spinning streamer's overlay.*

8. Undo / Revert Engine

Every state mutation — automatic or manual — pushes a reversible action to a per-user history stack. Linked actions from the same Kick event are grouped so reverting one reverts all.

8.1 Undo entry fields

Field	Type	Description
id	UUID	Unique action identifier
type	string	countdown:add, goals:add, roulette:result, toggle, style, preset
source	object	{ trigger: "auto"|"manual", event?, user?, coopSource? }
forward	object	What was applied (e.g., { seconds: 150 })
reverse	object	What to apply on revert (e.g., { seconds: -150 })
children	UUID[]	IDs of linked child actions from the same event
parent	UUID?	Parent action ID (if this is a child)
reverted	boolean	Whether this action has been undone

8.2 Co-op undo behavior

When a shared action is reverted (e.g., undo a co-op countdown addition), the revert affects the shared state and broadcasts to all co-op members. Each streamer sees the revert in their own undo panel. Only the streamer who received the original event (or their mod) can revert it — the co-op partner cannot undo events from the other channel.

9. Alert Queue System

Per-user FIFO queue for sequential alert playback. Alerts are always personal — they never cross co-op boundaries. Even when co-op is active, a sub alert for Streamer A only appears on Streamer A's overlay.

9.1 Alert types

Event	Template	Duration	Animation
Gifted subs	Name + count + confetti	5 seconds	Bounce drop
New sub	Name + welcome	4 seconds	Scale pop
Follow	Name + thanks	3 seconds	Slide up
Kicks gift	Name + gift + amount	5 seconds	Glitch
Goal reached	Goal name + celebration	6 seconds	Scale pop
Co-op goal reached	Goal + "via Partner"	5 seconds	Celebrate
Roulette result	Result text	4 seconds	Scale pop

10. Overlay Rendering Spec

10.1 GPU constraints

The overlay page shares GPU memory with the game capture in OBS. Only these CSS properties may be animated:

- transform (translate, scale, rotate) — GPU-composited
- opacity — GPU-composited
- clip-path — composited on most GPUs
- filter:blur() — moderate cost, max 1 simultaneous

> *NEVER animate: width, height, top, left, margin, padding, background-color, border, font-size. Use will-change on animating elements, reset to auto when idle. Max 3 simultaneous animations.*

10.2 Animation presets

Name	Enter	Exit	Cost
Fade	opacity 0→1, 300ms	opacity 1→0	Minimal
Slide up	translateY(30px)→0 + fade	translateY(-15px) + fade	Minimal
Scale pop	scale(.7)→1, spring ease	scale(.9) + fade	Minimal
Bounce drop	translateY(-40px)→0, bounce	translateY(20px) + fade	Minimal
Typewriter	clip-path reveal L→R	reverse	Minimal
Blur in	blur(8px)→0 + fade	blur out	Moderate
Glitch	clip-path slices + X jitter	reverse	Moderate
Celebrate	scale pop + rotate wobble	scale + fade	Minimal

11. Dashboard UI Spec

11.1 Desktop layout

Dark-mode control surface. Charcoal #0D0D0D base, Kick green #53FC18 accent. Five zones:

Zone	Width	Content
Top bar	100%	User avatar/name, Kick connection status, co-op status, preset switcher, undo button
Left panel	280px	Event log, alert queue, undo history, co-op panel
Center	flex	16:9 preview canvas, draggable elements
Right panel	300px	Element properties, countdown panel, goals panel, roulette panel
Bottom dock	100%	Toggle strip, scene presets, timer display, goal progress mini-bar

11.2 Mobile layout

Single column. Small 16:9 preview. Tab bar: Toggles, Timer, Goals, Wheel, Events, Co-op, History. Fixed bottom bar with presets + undo. Every touch target 44px minimum.

11.3 Mod dashboard variant

When a mod is logged in, the top bar shows a streamer selector dropdown. The mod sees the same dashboard but with restricted permissions: no co-op creation, no auth settings, no config deletion. Visual indicator (banner) shows "Managing: StreamerName's overlay".

12. Database Schema

Six tables in Supabase PostgreSQL. See the agent.md file for the complete SQL migration.

Table	Purpose
users	Kick OAuth profiles. kick_user_id, username, avatar, channel_slug, role, tokens.
mod_assignments	Links mods to streamers. streamer_id, mod_id.
overlay_configs	One per streamer. Full overlay state as JSONB. Versioned.
coop_sessions	Links two streamers. Holds shared_countdown and shared_goals JSONB.
coop_members	Members of a co-op session. share_countdown and share_goals toggles.
event_log	All Kick events for replay and analytics.
undo_history	Reversible action entries with parent/child links.

13. Deployment Strategy

13.1 Local development

24. Clone repo, npm install, create .env from .env.example
25. Create Supabase project, run migration SQL
26. Create Kick developer app, set callback to http://localhost:3000/api/auth/callback
27. Run node server.js, use ngrok for Kick webhooks
28. OBS Browser Source: http://localhost:3000/overlay/YOUR_USER_ID

13.2 Vercel production

29. Push to GitHub, connect to Vercel
30. Set all env vars in Vercel dashboard
31. Webhook endpoint: serverless function at /api/webhook/:userId
32. Real-time: Supabase Realtime subscriptions (replaces raw WebSocket)
33. Update Kick app callback URL to Vercel domain
34. Each streamer's OBS points to https://your-app.vercel.app/overlay/:userId

13.3 Environment variables

Variable	Description
KICK_CLIENT_ID	From Kick developer portal
KICK_CLIENT_SECRET	From Kick developer portal
SESSION_SECRET	Random 64-char string for express-session
SUPABASE_URL	Supabase project URL
SUPABASE_ANON_KEY	Supabase anonymous/public key
SUPABASE_SERVICE_KEY	Supabase service role key (server-side only)
BASE_URL	https://your-app.vercel.app (for OAuth redirect)
PORT	3000 (local only)

14. Build Phases

Six phases, ~22 sessions total. Each phase produces a testable deliverable. See agent.md for exact step-by-step instructions.

Phase	Deliverable	Priority	Sessions	Depends on
1	Server scaffold + Supabase + Auth (Kick OAuth + sessions)	**P0**	3	—
2	Overlay + Dashboard shell + WebSocket + State manager	**P0**	3	Phase 1
3	Kick webhooks + Undo engine + Alert queue	**P0**	3	Phase 2
4	Countdown + Goals + Co-op sync engine	**P0**	4	Phase 3
5	Roulette + Dashboard panels + Undo panel + Style editor	**P1**	4	Phase 4
6	Mobile + Vercel deploy + Supabase Realtime + Extras	**P1**	3	Phase 5
	TOTAL		~22	

> *The agent.md file contains the exact build order with 20 numbered steps, the complete database migration SQL, file structure, and implementation notes for every module. Hand both this PRD and the agent.md to the development team or AI coding agent.*

End of PRD | StreamDeck v3.0 | March 2026 | Confidential